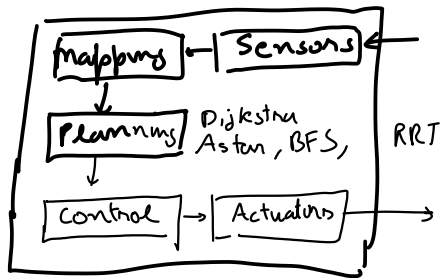


Control

Mapping



Robotic arm
 $R, T \begin{cases} \text{Forward Kinematics (FK)} \\ \text{Inverse Kinematics (IK)} \end{cases}$

Planning : Going from start state to goal state

Higher-level of abstraction
Long trajectories

Trajectory in terms of states



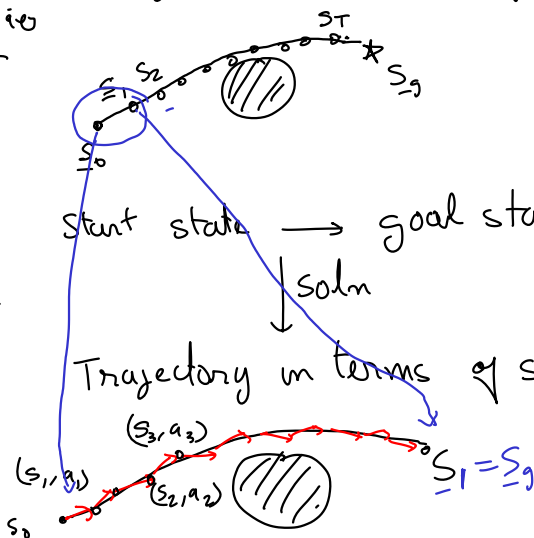
MEE 444
 Robotic control

Control : Start state $\rightarrow$ goal state

Lower level of abstraction

Trajectory in terms of state-action pairs / policy function
 state feedback control

Follow the trajectory provided by the Planner



Control

- PID : Proportional - Integral - Differential controller $\begin{pmatrix} P \\ PI \\ PD \end{pmatrix}$
- LQR : Linear Quadratic Regulator $\rightarrow$ iLQR incremental LQR
- MPC : Model Predictive control
- RL : Reinforcement Learning $\rightarrow$ Q-learning (Tabular form)

PID controller

single dimensional control

for multi dimensional control use one PID per dimension

For Jetbot: what is the state space, action space, system dynamics, and cost function?

We only covered Discrete planning

But Jetbot has states and action in cont. domain.

Simplified mathematical models for robot like Jetbots

↳ Unicycle model ✓

↳ Ackermann drive



$$\text{state} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} = \underline{s}_t$$

$\underline{s}_t \in \mathcal{S}$
 $\uparrow$
 all possible
 values of
 state
 = State space

$$\underline{a}_t = \text{action} = \begin{bmatrix} v_t \\ \omega_t \end{bmatrix} \begin{matrix} \text{linear velocity} \\ \text{angular velocity} \end{matrix}$$

$\uparrow$
control signal (u_t)

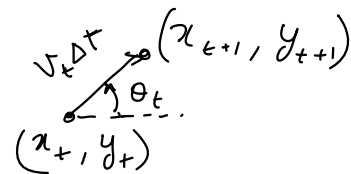
$\underline{a}_t \in \mathcal{A}$
 $\uparrow$
 Action space

$\underline{u}_t$ RL $\underline{a}_t$

System dynamics (discrete time, Δt)

$$\underline{s}_{t+1} = f(\underline{s}_t, \underline{a}_t)$$

$$\underbrace{\begin{bmatrix} x_{t+1} \\ y_{t+1} \\ \theta_{t+1} \end{bmatrix}}_{\underline{s}_{t+1}} = \underbrace{\begin{bmatrix} x_t + v_t \Delta t \cos \theta_t \\ y_t + v_t \Delta t \sin \theta_t \\ \theta_t + \omega_t \Delta t \end{bmatrix}}_{f(\underline{s}_t, \underline{a}_t)}$$



Acceleration

Markovian state

$$\underline{s}_t = \begin{bmatrix} x_t \\ y_t \\ v_t \\ \theta_t \end{bmatrix}$$

must capture all the info

$$\underline{a}_t = \begin{bmatrix} b_t \\ \omega_t \end{bmatrix} \begin{matrix} \text{linear acc} \\ \text{ang. vel} \end{matrix}$$

$$\underline{s}_{t+1} \perp \underline{s}_{t-1} \mid \underline{s}_t$$

$\underline{s}_{t+1}$ is conditionally ind. of $\underline{s}_{t-1}$ given $\underline{s}_t$

that is required to predict next state

cost function = defines the task

$$c(s_t, a_t) = \|s_t - s_g\|^2 + \lambda \|a_t\|^2 \quad \leftarrow \text{example}$$

Control problem

$$\{a_t\}_{t=1}^T = \underset{\{a_t\}_{t=1}^T}{\text{minimize}} \quad \sum_{t=1}^T c(s_t, a_t)$$

$$\text{subject to} \quad s_{t+1} = f(s_t, a_t) \\ \forall t \in \{1, \dots, T\}$$

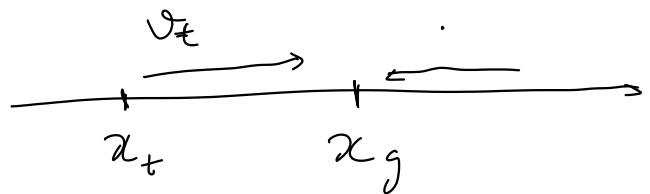
policy function $\uparrow$

$$\pi^*(s_t) = \underset{\pi}{\text{minimize}} \quad \sum_{k=t}^T c(s_k, \pi(s_k))$$
$$\text{subject to} \quad s_{k+1} = f(s_k, \pi(s_k)) \\ \forall k \in \{t, \dots, T\}$$

PID controller does not use system dynamics or the cost function. Hence it is not called optimal control

We construct an error function (must be signed)

$$e(x_t) = x_g - x_t$$



$$a_t = \underbrace{k_p e(x_t)}_{\text{Proportional term}} + \underbrace{k_I \int_0^t e(x_t) dt}_{\text{Integral}} + \underbrace{k_d \cdot \frac{d}{dt} e(x_t)}_{\text{Differential term}}$$